STRATEGIC DECISION-MAKING IN CLOUD RESOURCE ALLOCATION

Game Theory Project Report

Course: CSE 3145 — Game Theory
Institution: Manipal University Jaipur

Group Members: Aditya Chauhan and Utkarsh Mehra
Faculty: Ms Shweta Sharma
Submission Date: 26/10/2026

Abstract

Cloud computing providers must allocate finite CPU and memory resources among competing application requests. Different allocation rules can lead to different outcomes, especially when applications have different priorities or users act strategically. This project implements a small Python simulator that compares First-Come, First-Served (FCFS) with Priority-Based Allocation and calculates resource utilization and request acceptance. It also presents a two-player normal-form game in which two cloud tenants choose Low Bid or High Bid for priority. The application displays the payoff matrix and calculates pure-strategy Nash equilibria. The prototype is intended for educational demonstration of strategic decision-making and is not connected to a production cloud platform.

1. Problem Statement

Cloud environments have limited computing capacity, while applications may request different amounts of CPU and RAM. An allocation policy must decide which requests to accept without exceeding capacity. In addition, tenants competing for scarce priority may choose actions based on the expected choices of other tenants. The problem is to simulate allocation decisions and illustrate how strategic incentives can affect outcomes.

2. Objectives

- Implement FCFS and Priority-Based resource allocation strategies.
- Enforce CPU and RAM capacity constraints for every accepted request.
- Calculate CPU utilization, RAM utilization, accepted/rejected requests, and acceptance rate.
- Compare both allocation policies using a common sample workload.
- Model two tenants' strategic choices using a normal-form payoff matrix.
- Identify pure-strategy Nash equilibria by checking mutual best responses.
- Present the results in an interactive dashboard suitable for demonstration.

3. Scope and Assumptions

- The system is a simulator; no real cloud resources are provisioned.
- CPU and RAM are treated as divisible integer quantities requested by each application.
- An application is accepted only when both its CPU and RAM requirements fit the remaining capacity.
- Resources remain allocated for the duration of the simulation; there is no release or time scheduling.
- Priority values are 1 = Low, 2 = Medium, and 3 = High. Ties preserve original request order.
- The game-theory payoff values are illustrative utility points chosen for teaching; they are not collected from a real provider.

4. Methodology

4.1 FCFS Allocation

The FCFS strategy considers requests in the order supplied. For each request, it checks whether both CPU and RAM are available. If both conditions are satisfied, the request is accepted and its resources are deducted from the remaining capacity; otherwise, it is rejected.

4.2 Priority-Based Allocation

The priority-based strategy sorts requests from highest to lowest priority. Requests with equal priority retain their original order. The same capacity checks used by FCFS are then applied. This policy can favour important applications, although it may reject a later low-priority request even when that request could otherwise fit.

4.3 Performance Metrics

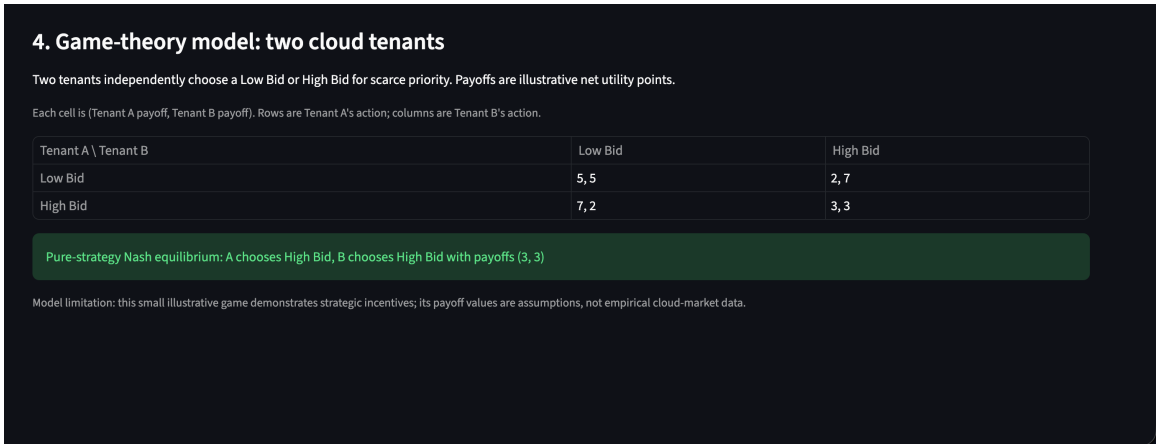
Metric	Formula / Meaning
CPU utilization	Allocated CPU / Total CPU × 100%
RAM utilization	Allocated RAM / Total RAM × 100%
Acceptance rate	Accepted requests / Total requests × 100%
Rejected requests	Total requests − Accepted requests

4.4 Game-Theory Model

Two tenants, A and B, independently choose Low Bid or High Bid to compete for scarce priority. Each outcome gives a pair of illustrative utility payoffs (A, B). A pure-strategy Nash equilibrium is an action pair in which neither tenant can improve its payoff by changing its own action alone while the other tenant's action remains fixed.

Tenant A \ Tenant B	Low Bid	High Bid
Low Bid	(5, 5)	(2, 7)
High Bid	(7, 2)	(3, 3)

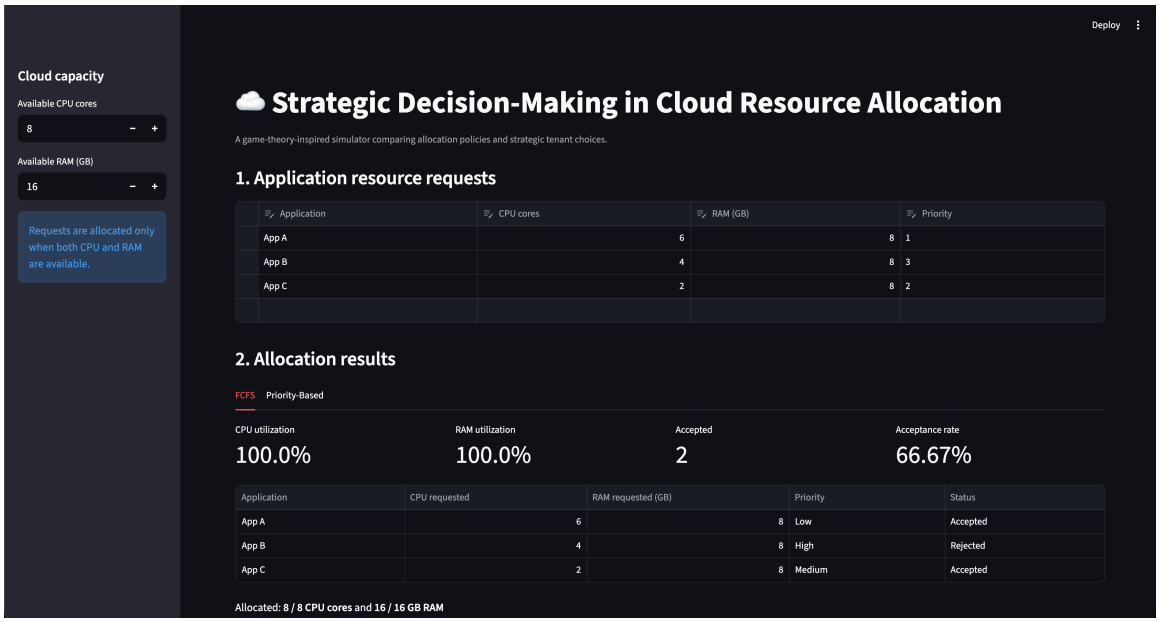
For either tenant, High Bid yields a higher payoff than Low Bid regardless of the other tenant's action: against Low Bid, the payoff is 7 rather than 5; against High Bid, it is 3 rather than 2. Thus High Bid is a strictly dominant strategy for each tenant, and (High Bid, High Bid) is the unique pure-strategy Nash equilibrium, with payoffs (3, 3).



5. System Design and Implementation

The project is implemented in Python. Streamlit provides the browser-based interface; Pandas manages the editable request table and metrics; Matplotlib generates the strategy comparison chart.

File	Purpose
algorithms.py	FCFS and priority-based allocation logic
metrics.py	Utilization and acceptance calculations
game_theory.py	Payoff matrix and pure Nash equilibrium calculation
main.py	Command-line demonstration
app.py	Interactive Streamlit dashboard
requirements.txt	Python dependencies



6. Sample Input

Application	CPU cores	RAM (GB)	Priority
App A	6	8	Low (1)
App B	4	8	High (3)
App C	2	8	Medium (2)

Sample cloud capacity: 8 CPU cores and 16 GB RAM. The workload includes a low-priority, CPU-heavy App A; a high-priority App B; and a medium-priority App C.

7. Results and Analysis

Strategy	CPU Utilization	RAM Utilization	Accepted	Rejected	Acceptance Rate
FCFS	100.0%	100.0%	2	1	66.67%
Priority-	75.0%	100.0%	2	1	66.67%

Based

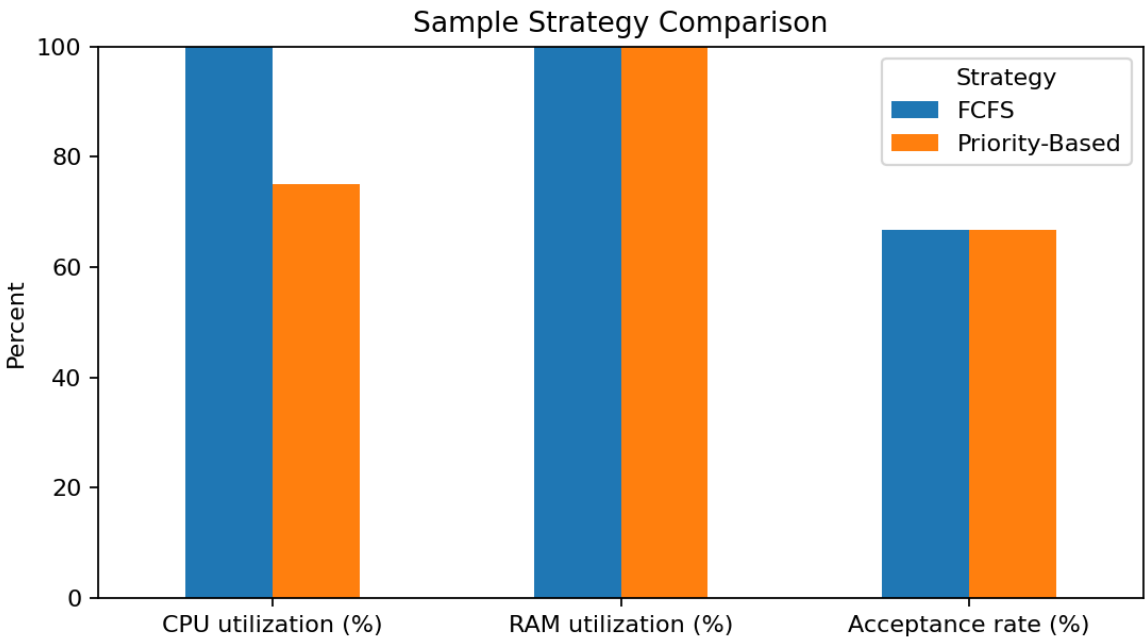
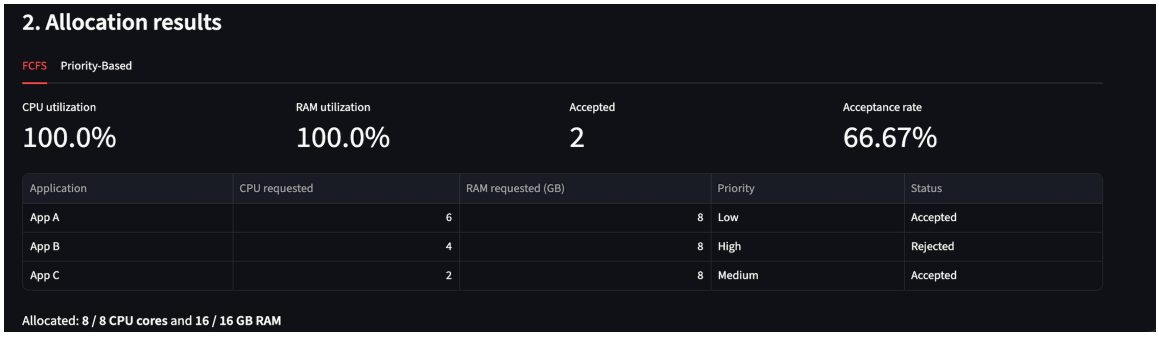


Figure 1. Comparison chart generated from the bundled sample workload.





Interpretation: FCFS accepts App A and App C, while Priority-Based allocation considers App B first and accepts App B and App C. Both accept two of three requests and use all RAM, but CPU utilization differs (100% for FCFS versus 75% for Priority-Based). This illustrates a trade-off: priority can favour

important work even when it does not maximize CPU utilization. Capture screenshots from the running dashboard before submission.

8. Testing

Test	Scenario	Expected result
T1	Requests fit within capacity	All requests are accepted.
T2	Combined requests exceed capacity	At least one request may be rejected.
T3	High-priority request appears later	Priority strategy considers it before lower-priority requests.
T4	One request exceeds total RAM	That request is rejected.
T5	Empty request list	No crash; metrics should be zero if passed to metric calculation.

9. Applications

- Teaching how resource allocation policies influence cloud workloads.
- Comparing scheduling policies in a simplified virtualized environment.
- Demonstrating strategic interaction among tenants competing for scarce service priority.
- Supporting classroom discussions of utility, best responses, and Nash equilibrium.

10. Limitations

- No live cloud provider, virtualization layer, or real billing system is connected.
- The model does not include task duration, deadlines, network bandwidth, energy, or resource release.
- The game payoff matrix is illustrative and would need empirical calibration for real-world use.
- Utilization and acceptance rate alone do not fully measure fairness, latency, or service quality.

11. Future Scope

- Add request arrival times and release resources when jobs finish.
- Introduce a third allocation policy, such as best-fit or weighted priority.
- Add waiting time, latency, fairness, and energy consumption metrics.
- Allow users to edit game payoffs and visualize best responses.
- Connect the simulator to a cloud test environment only if required and permitted.

12. Conclusion

The prototype demonstrates two deterministic allocation strategies and a simple strategic game relevant to cloud resource competition. It enforces CPU/RAM limits, reports allocation metrics, and provides an interactive comparison. The game-theory component links the software demonstration to strategic choices, payoffs, best responses, and Nash equilibrium. The project shows that allocation

policy affects outcomes and that the most appropriate strategy depends on both system constraints and the objective being optimized.

Appendix A: Group Contribution

Member	Contribution
Member 1: Aditya Chauhan	Allocation algorithms, sample workload, code testing, algorithm documentation.
Member 2: Utkarsh Mehra	Metrics, dashboard, graphs, results analysis, report/PPT compilation.